Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Maze Lab: Interactive Maze Generation and Comparative Algorithmic Pathfinding

Team Name: *Team #*	*Team Blockbuster* *T272*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Kumar, Tanmay – 2510370439* *2510370439@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Saxena, Sparsh – 2510370313* *2510370313@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Sethi, Hemang – 2510370341* *2510370341@geu.ac.in* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Most core computer science courses teach data structures and algorithms through standalone coding tasks or static textbook diagrams[cite: 16]. Consequently, students often struggle to grasp how components like graph traversals, disjoint sets, or priority queues interact inside a single software application[cite: 16].

Maze Lab directly addresses this by offering a hands-on environment where players first navigate a generated maze manually and then evaluate their path against automated solvers[cite: 16]. Combining object-oriented system design with graph theory gives learners a direct, practical look at runtime performance, memory overhead, and heuristic search behavior[cite: 16].

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Current learning tools usually split into simple text-only programs or web-based pathfinding visualizers[cite: 16]. Text exercises lack clear graphic feedback, while web visualizers rarely let users control a character through the maze[cite: 16]. Additionally, many existing codebases hard-code their algorithms, making it tricky to add or swap new solvers[cite: 16]. Maze Lab solves these issues by combining player controls with graph solvers using a clean, interface-driven C++ layout[cite: 16].

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Our main goal is to build a flexible C++ application that generates valid mazes, supports manual player controls, and compares automated solving algorithms side by side[cite: 16].

Key milestones for implementation:

1. **Architecture Setup:** Define base `Cell` and `Maze` models alongside abstract `MazeGenerator` and `MazeSolver` interfaces[cite: 16].
2. **Maze Generation:** Build an iterative Randomized DFS generator and a Kruskal's algorithm generator backed by Union-Find[cite: 16].
3. **Player Mechanics & Console UI:** Implement player navigation, wall collision detection, and ASCII terminal rendering[cite: 16].
4. **Pathfinding Solvers:** Construct BFS and A* pathfinders that collect step counts, visited cell totals, and runtime metrics[cite: 16].
5. **Validation & Delivery:** Perform integration testing across all components, build a side-by-side benchmark mode, and complete final documentation[cite: 16].

Project Approach (3 pts)

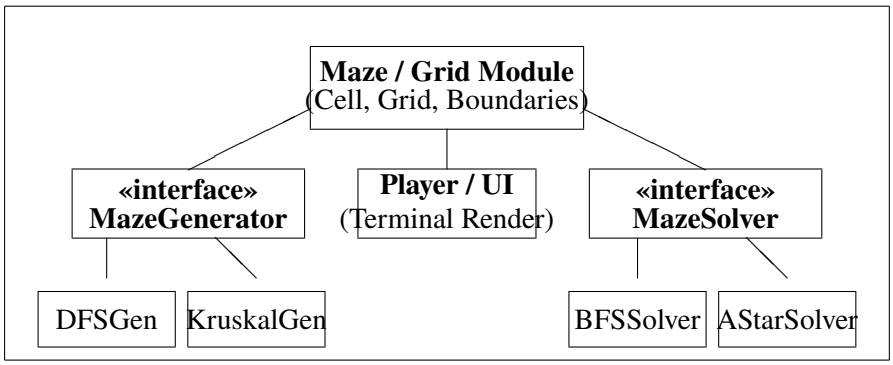
(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

We are writing the program in C++ with an emphasis on modular object-oriented design[cite: 16]. The maze itself sits in a 2D grid of `Cell` structures, where removed walls correspond to graph edges[cite: 16].

To keep algorithms decoupled, we use abstract base classes under the Strategy Pattern[cite: 16]. `MazeGenerator` handles generation, while `MazeSolver` manages pathfinding[cite: 16]. Maze generation relies on explicit-stack DFS alongside Kruskal's algorithm built on a Disjoint Set Union class[cite: 16, 17]. Pathfinding tasks use BFS for shortest paths and A* guided by Manhattan distance heuristics[cite: 17]. Player logic lives separately in a `Player` class to verify movement before updating position[cite: 17]. Terminal output handles primary rendering, while SDL graphics remain an optional secondary goal[cite: 17].

System Architecture (High Level Diagram) (2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

Key outcomes include a complete C++ terminal program, well-documented source code with clean commit records, automated unit tests, technical project documentation, and a live demonstration comparing human player paths against BFS and A* solutions[cite: 17].

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

We assume the maze grid dimensions remain fixed during solving and that generated mazes are always simply connected with no loops[cite: 17]. We also assume the terminal host environment supports standard C++ I/O and ANSI color sequences[cite: 17].

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. Cormen, T. H., et al. *Introduction to Algorithms*. MIT Press[cite: 17].
2. Gamma, E., et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley[cite: 17].
3. ISO/IEC C++17 Standard Specifications[cite: 17].